Docker with SpringBoot

1. Dockerizing an Exisiting Application

- To Dockerize an existing application, we can use **Docker file.**
- A dockerfile is a **text file containing instructions/commands.**
- These instructions are used by **Docker Daemon** to build **Docker Image.**
- A Dockerfile can specify
  - Base Image
  - Application code
  - Dependencies
  - Configuration
  - Commands to execute

2. Example Docker file

```
From openjdk:11
VOLUME /tmp
ADD target/my-app.jar my-app.jar
EXPOSE 8080
ENTRYPOINT [“java”, “-jar”,”/my-app.jar”]
```

FROM :-

- uses an official OpenJDK 11 image from **Docker Hub** as the **base image**
- The base image contains what is needed to **run** a **java application.**

VOLUME :-

- Creates a **mount point** for an externally **mounted volume.**
- The example uses /tmp

ADD :-

- Adds the **application's JAR** file from the **target** directory **into Docker Image**
- The JAR is named :- myapp.jar

EXPOSE :-

- Exposes port **8080**
- This allows the application running **inside** the **container** to be accessed from **outside** the **container.**

ENTRY :-

- Specifies the **command** to execute when the **container starts.**
- in this example, it runs the JAR file.

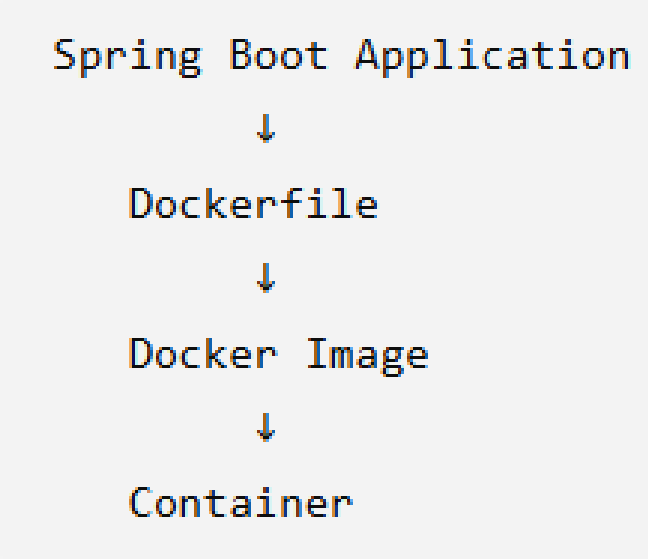
3. Docker + SpringBoot

There are two approaches for creating **Docker Image :-**

- Approach 1 – Dockerfile
- Approach 2 – SpringBoot Maven plugin

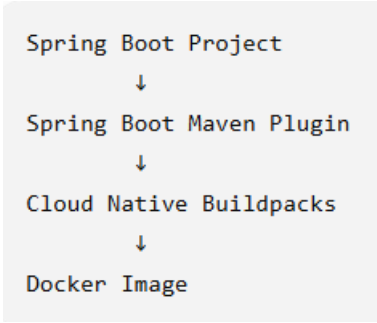
Approach 1 – Docker File

- Create a Dockerfile in the **root directory** of the project
- Define the required instructions **manually**
- Docker uses the **Dockerfile** to create the image



Approach 2 – Spring Boot Maven Plugin

- Spring Boot provides a **Spring Boot Maven Plugin**
- The plugin can be used to create a **Docker Image without** manually writing a **Dockerfile.**
- The process is **automated** based on the project's **dependencies** and **configuration**.

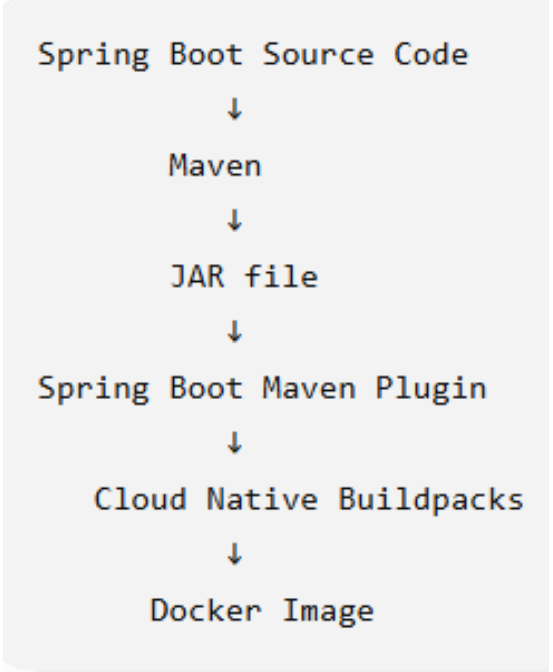


4. Cloud Native Buildpacks

- SpringBoot **internally** use **Cloud Native Buildpacks** to create Docker Images.
- Cloud Native Buildpacks are associated with **Cloud Native Computing Foundation.**
- Build packs are designed to create **container images** from source code **without requiring a Dockerfile.**
- Buildpacks **analyze** the project.
- They determine the appropriate
  - Tools
  - Libraries
  - JDK/runtime requirements
  - Dependencies
- They then package what the **application needs** into the **Docker Image**

5. Spring Boot Maven Plugin + Build Packs

- The Spring Boot Maven Plugin **triggers** the Buildpack process.
- The plugin first **packages** the **application** as a **JAR**
- It then hands the **application** over to **Buildpacks.**
- **Buildpacks** create **Docker Image.**



6. Layering

- Buildpacks uses a technique called **layering**
- Instead of treating the entire **Docker Image** as **one large unit**, the image is **divided** into **separate layers.**
- for Example :-

```
Docker Image
|
├── JVM layer
├── Application dependencies layer
└── Application layer
```

- Docker can cache these layers separately
- If you make a small change to your application:
  - Unchanged layers can be reused.
  - Only the changed layer needs to be rebuilt.
- This makes subsequent Docker Image builds faster.

Example:

```
First Build
____________________

JVM              → Build
Dependencies     → Build
Application      → Build


Second Build
____________________

JVM              → Reuse cache
Dependencies     → Reuse cache
Application      → Rebuild
```

7. Paketo Buildpacks

- SpringBoot team works closely with the **Paketo Buildpacks project.**
- Paketo Buildpacks are **implementations** of the **Cloud Native Buildpacks specification.**
- They provide:
  - JVM support
  - Tools required to **compile/package** applications
  - Support for creating Docker Images for SpringBoot applications

8. Final Result

After the Buildpacks process:

- The result is a **Docker Image**
- The image can run in a Docker-compatible runtime
- The image contains what the application needs, including:
  - JVM
  - Application Dependencies
  - Application Itself

9. Advantages of SpringBoot + Buildpacks

- **No Dockerfile required**
  - Buildpacks automatically handle many tasks that would normally be written in a Dockerfile.
- **Sensible Defaults**
  - Buildpacks provide sensible defaults for various project types, including SpringBoot
- **Consistent Environment**
  - The application can run using the environment packaged during the image-building process.
- **Security**
  - Buildpacks can keep their base images and libraries updated
  - This helps to maintain security
- **Layering and efficiency**
  - Separate layers can be cached and reused
  - If only one part changes, unchanged layers don’t need to be rebuilt
  - This can significantly speedup subsequent image builds.

Most Important Flow to Remember

Traditional Dockerfile approach

```
Spring Boot App
      ↓
 Dockerfile
      ↓
Docker Image
      ↓
Container
```

Spring Boot Buildpacks approach

```
Spring Boot App
      ↓
Maven Plugin
      ↓
Cloud Native Buildpacks
      ↓
Paketo Buildpacks
      ↓
Docker Image
      ↓
Container
```

Interview-Ready Explanation

- A Spring Boot application can be Dockerized either by
  - creating a Dockerfile manually
  - or by using the Spring Boot Maven Plugin with Cloud Native Buildpacks.
- With the Buildpacks approach,
  - the project is analyzed automatically,
  - the required runtime,
  - dependencies
  - and application are packaged into an image,
  - and layering allows Docker to cache unchanged parts of the image for faster subsequent builds.